Requirements Document

Author(s)- Toshura Washington, Nicole Band, Devran Turson, Darrin Annor, Kripa Bishwakarma

Introduction

The purpose of this document is to outline the requirements for the “Space Cats” project.

Designed for players who enjoy adventure and strategic gameplay within a space exploration setting, Space Cats is a top-down roguelike game where the player assumes the role of a cat whose goal is to rescue his kidnapped parents from the rat villain. The player is given a finite number of lives and must progress through increasingly challenging levels with careful planning and calculated decision making in order to defeat the Rat King.

1. Functional Requirements (things the game MUST DO)

1.1 Home page

- The home page will display the game title, Space Cats.
- The home page will display a “start game”.
    - Click "start game" to start the game
- The home page will show a demo of gameplay.
- Upon startup, the background story and controls will be displayed on the screen:
    - Controls at the bottom of the screen
    - The story will have music and be presented as slides

1.2 Game Page

- The game page will display 3 different cat characters the player can choose from.
- The game must have 4 planets:
    - Nacho
        - Fire/Lava Planet
    - Blue Cheese
        - Poison Planet
    - Parmesean
        - Snow Planet
    - CheeseCake

            - Boss Planet
- The game page must display the UI elements.
    - Health bar
        - Red
    - Inventory
        - 3 slots
            - Can be 1-3 potions or weapons
    - Level/planet
    - Remaining lives
- The player will navigate through different planets:
    - Maps navigated through halls and dungeon rooms
        - After the Boss, the Rocket will land for us to leave
    - Loading screen after every planet to show progression
        - A spaceship going through space
- When the player loses all their lives, the screen will display a “Game Over” screen.

1.3 Game Setup

- The player’s inventory must be cleared.
- The set level must be reset to a default state.
- The player must spawn in its designated locations, their spawnpoint, in each planet.
- Enemies must spawn in its designated location in each planet.
- Chest with the weapons and potions are randomly generated throughout the level.

1.4 Player Mechanics

- **Movement:**
    - When the user clicks the arrow keys or WASD keys the player sprite faces the designated direction and enters idle animation.
    - When the user holds the arrow keys or WASD key the player sprite enters moving animation and moves the sprite toward the designated direction.
    - 8 direction movement:
        - Combination of neighboring keys will result in a 45 degree movement.
            - Ex. W or (UP) + A or (LEFT) = movement towards the northwest direction.
- **Combat**
    - Players attack by hitting the space bar.
    - Players aim long range weapons with the mouse cursor.
    - Players can drink potions to boost their health or attack to last longer in the game.
    - Players lose health when hit by the enemy.
    - Enemies lose health when hit by the player.

    - Enemies must be defeated and removed from play when their health reaches zero.
    - Player is respawned at its spawn point when health reaches zero.

1.5 Enemy Mechanics

- Enemies spawn randomly around the map at the start of each level.
- Enemies stay in idle animation till they enter the user's line of sight.
- Enemies move towards the player in a straight line when in the user's line of sight.
- Enemies sprite changes depending on its relation to the player.
    - ex) when the enemy sees the player it will be on attack mode, else, it will stay on idle mode.

1.6 Inventory Handling

- Players can collect, equip, use, and drop items.
- Max inventory space of 3.
    - Game must prevent player from equipping more than 3 items
- If the player tries to equip another item whilst inventory is full, the game will give the player the option to drop one item in order to add the new item.

1.7 Level Progression Conditions

- Players can only progress to the next planet (level) if all enemies are defeated.
    - The rats and the rat king
- Difficulty will be progressive
    - Enemy stats rise with the progression between levels.
        - Ex: enemy health rises from 100HP to 125HP after completing the first level
- The player will start at a random planet.
- Once the player defeats the last planet, the player wins and can rescue their parents.
    - The game will display a “Victory” screen
- Player will have 3 lives per game.
- Once all lives are used, the player will restart from the first planet.
- The game will have the option to play the game again.

1.8 Character Customization (Optional Feature)

- Player can customize the appearance of their character.
- Does not affect game play conditions / mechanics.
- (Implemented only if there is remaining development time)

2. Nonfunctional Requirements (other properties the system must have)

2.1 Performance

- The game shall load quickly when initiated.
- The game shall respond to user inputs without delay.
- The game shall handle the multiple event executions simultaneously without noticeable delay.
    - Ex. Simultaneous movement of multiple enemies towards the player

2.2 Usability

- The UI shall be easy to understand to be handled.
- The game shall contain an “instructions” tab with keybinds and any clear instructions required for the user.
- The game shall provide clear feedback to the user.
    - Ex. sound effects, change of brightness / color

2.3 Accessibility

- The game shall be played with a keyboard and mouse / trackpad.
- The game shall maintain a clear and readable color contrast between texts and backgrounds.
- The game shall avoid low-contrast color combinations to make text difficult to read.
- The game shall provide clear visual feedback for player actions.
    - Ex. change in health bar, decrease of lives ect.

3. Technical Requirements

- Game developed using JavaScript and p5.js
- Must run on a modern web browser.
- Must support keyboard and mouse input for player movements, attacks and aim.